

Chassis Physics and Space Planning

The Chassis Is the Robot's Foundation

The chassis is the base of the robot. It supports the robot's weight, holds the wheels and motors, protects the electronics and battery, and creates space for future mechanisms.

A good chassis is not just strong. It also needs to be useful. The shape of the chassis affects how the robot drives, turns, carries weight, handles contact, and makes room for attachments like a shooter, hopper, trigger, or intake.

The Starting Size Constraint

FTC robots usually need to fit inside an **18 inch x 18 inch x 18 inch** starting size. This means the robot must begin the match inside that size limit.

That does not mean every robot should use the full space. A larger robot may have more room for mechanisms, but a smaller or narrower robot may move through tight spaces, park, or share the field more easily.

For the DECODE-style challenge in this course, teams should think about:

- Can the robot fit at the start?
- Can it park or share space with other robots?
- Can game pieces move around, into, or through the chassis?
- Is there room for the battery, electronics, shooter, hopper, and trigger?
- Can the team reach important parts for repair?

Supporting Loads

The chassis carries everything else on the robot. If the frame bends, twists, or comes loose, every other system becomes less reliable.

The chassis needs to support:

- motors,
- wheels,
- battery,
- control hub and electronics,
- wires,
- shooter or scoring mechanism,
- hopper or game-piece storage,
- and any extra brackets or attachments.

Student-friendly idea:

A chassis is like a table. If the table wobbles, anything built on top of it becomes harder to use.

Staying Connected

A robot frame is made from parts connected by screws, nuts, brackets, plates, and connectors. The parts need to stay connected while the robot drives, turns, stops, bumps into objects, and carries weight.

Common chassis problems include:

- screws loosening,
- brackets bending,
- frame corners twisting,
- wheels rubbing,
- wires getting pulled,
- attachments shaking loose,
- or the battery sliding around.

Good teams check connections after testing. A robot that works once but falls apart after five minutes is not reliable yet.

Withstanding Contact

Robots may bump into game objects, field walls, or other robots. The chassis should be strong enough to handle normal contact without losing shape.

Questions to ask:

- Are important parts protected?
- Are wires away from wheels and moving parts?
- Are battery and electronics secure?
- Are corners strong enough?
- If the robot pushes a ball or artifact, does the frame twist?
- If another robot bumps us, will our mechanism stay aligned?

This does not mean the robot must be heavy or overbuilt. It means the frame should be strong enough for the job.

Motor Placement and Turning

Where the drive motors and drive wheels are placed changes how the robot feels to drive.

If the drive wheels are closer to the middle of the robot, the robot may turn more easily and may be able to rotate in place more smoothly. This can make the robot easier for a human driver to control.

If the drive wheels are farther apart or closer to the ends, the robot may feel more stable, but it may take more space to turn. It may also be harder to line up quickly.

Motor placement affects:

- turning radius,
- ability to rotate in place,

- how easy the robot is to control,
- how much space remains inside the frame,
- where the battery and electronics can go,
- how easy it is to add attachments later,
- and how protected the motors are.

There is no perfect motor placement. Teams need to decide what matters most: control, space, strength, repair access, or future mechanisms.

Driving in Multiple Directions

Most course robots will use two powered drive wheels and two support wheels, often omni wheels. This kind of robot can drive forward and backward and turn. It usually cannot slide sideways like a competition robot with mecanum wheels.

That means the chassis shape and motor placement matter. If the robot cannot move sideways, it must turn and line up carefully. A driver needs enough control to approach the goal, reach the human operator, avoid other robots, and park.

Clearance Off the Floor

Clearance means how much space there is between the bottom of the robot and the floor.

Too little clearance can cause:

- the robot to scrape,
- screws or brackets to catch on field elements,
- wheels to lose contact,
- or game pieces to jam under the frame.

Too much clearance can raise the robot's weight and make it less stable.

Good teams check whether the robot can move over the field surface without dragging or catching.

Space for Attachments

The chassis should leave room for future systems. A team may need space for:

- shooter,
- hopper,
- trigger servo,
- ball or artifact path,
- wiring,
- battery removal,
- electronics access,
- and repairs.

An open chassis may leave space for game pieces to enter or move through. A closed chassis may be stronger and easier to build. This is a trade-off.

Could Challenge: Add More Physics

If your team is ready for a challenge, you can explain the chassis using more formal physics ideas:

- **center of mass:** where the robot's weight is balanced,
- **torque:** turning force from motors or from weight placed far from the center,
- **friction:** grip between wheels and floor,
- **normal force:** how weight is shared across wheels,
- **stability:** how hard it is to tip the robot,
- **turning radius:** the space needed to complete a turn,
- **structural rigidity:** how much the frame resists bending or twisting.

You do not need these terms to build a first chassis, but they can help explain why one design works better than another.

Big Idea

The chassis is a set of engineering decisions. Shape, size, motor placement, clearance, and open space all affect how the robot works. A good chassis supports the robot, survives normal contact, leaves room for future mechanisms, and is easy enough to repair and improve.

Think About It

What should your team prioritize in the chassis: strength, open space, turning control, repair access, or room for future mechanisms?

DECODE Scoring and Strategy Background

What Do Teams Need to Know Before Choosing a Strategy?

In DECODE, two alliances compete at the same time. Each alliance has two robots. Robots try to score game pieces, build patterns, avoid penalties, and return to their base before time runs out.

This course uses a simplified version of the full FTC challenge. Your team will still use the real ideas of engineering strategy: choosing what to attempt, designing under constraints, testing with evidence, and explaining why your choices make sense.

The Big Idea

A strong robot is not always the robot that tries to do the most things. A strong robot is one that can do useful things reliably.

For this course, teams should think about four strategy questions:

- How will we score balls or artifacts?
- Where will we launch from?
- What can we do in autonomous?
- How will we help both alliance robots park or return to base?

Course Constraints

These constraints shape your strategy.

Constraint	What It Means
No intake	Your robot will not collect balls from the field. A human operator will load balls into the robot.
Hopper limit goal	Your hopper should be designed to carry up to three balls.
No AprilTag reading	Your robot will not use camera vision to identify field targets.
No distance sensors	Your robot will not automatically measure exact distance from the goal.
Limited build time	A simple reliable robot may outperform an unfinished complex robot.
Shared field space	Your robot should leave room for an alliance partner to move, score, and park.

These are not failures. They are engineering constraints. Good teams make smart choices inside real limits.

Match Periods

FTC matches have two main periods.

Period	What Happens
Autonomous	Robots run pre-programmed instructions without driver control.
TeleOp	Human drivers control the robots.

In the official DECODE game, autonomous can include moving off the launch line, scoring artifacts, and building a pattern. In this course, every team should try for a simple autonomous movement. Some teams may challenge themselves to attempt autonomous scoring if their robot is ready.

Scoring Ideas That Matter for This Course

The official DECODE game includes several ways to earn points. Your team does not need to master every official rule to make a strong strategy, but you do need to understand the parts that affect your design.

Scoring Idea	Why It Matters
Scoring artifacts or balls	Your launcher, hopper, driver control, and launch site all affect scoring.
Leaving the launch line in autonomous	A simple autonomous movement can create reliable early points.
Returning to base	Parking or returning to base can be a valuable endgame goal.
Two robots fully returned to base	Alliances can earn an additional bonus when both robots fully fit in base.
Ranking points	Some achievements can matter beyond match points because they help teams rank higher.
Penalties	A risky strategy can lose value if it creates penalties.

Why Parking and Robot Size Matter

In DECODE, both robots may need to return to the alliance base at the end of the match. If both robots can fully fit, the alliance can earn an additional bonus.

This makes robot size a strategy decision. A team might choose:

- a narrow robot that leaves room for an alliance partner,
- a compact robot that can park quickly,
- a taller or unusual robot shape that tries to share space differently,
- or a wider robot that is easier to build but may make parking harder.

The unusual choice is not automatically better. A narrow or tall robot may create problems with balance, hopper space, launcher placement, battery access, and driver control. The team should only choose that path if the parking benefit is worth the trade-off.

Human Operator Loading

Because this course does not include an intake, your human operator matters.

Your team should decide:

- where the human operator loads balls,
- how the robot lines up for loading,
- how many balls the hopper can safely hold,
- how the driver knows the robot is ready to leave,
- and how the team avoids unsafe or illegal loading.

A fast launcher does not help if the robot takes too long to load or if the hopper jams.

Launch Site Choice

Because your robot will not use AprilTag reading or distance sensors, your team needs a repeatable launch site.

A good launch site should be:

- legal,
- reachable,
- easy for the driver to recognize,
- far enough from traffic or defense,
- compatible with the launcher angle and power,
- and respectful of the alliance partner's path.

Your team should choose one first launch site and one backup launch site.

Autonomous Goals

Every team should try to create at least one autonomous program that moves the robot predictably.

Possible autonomous goals:

Goal	Difficulty	Why a Team Might Choose It
Drive off the launch line or starting area	Easier	Reliable first autonomous success.
Drive to a useful position	Medium	Sets up TeleOp or avoids traffic.
Park or move toward base	Medium	Supports a simple endgame or field-position plan.
Score autonomously	Hard	Strong challenge if the launcher and starting setup are consistent.

Autonomous scoring is exciting, but it is risky without sensors. Small changes in starting position, battery level, wheel traction, or ball placement can change the result.

Penalty Avoidance

Avoiding penalties is part of strategy. A team can lose the value of a good scoring plan by breaking rules, blocking illegally, loading unsafely, contacting where contact is not allowed, or continuing an action after time expires.

Teams should ask:

- What are the easiest penalties for us to accidentally commit?
- What should the driver avoid?
- What should the human operator avoid?
- What should the coach remind the team about?
- What part of our strategy needs a rules check before we rely on it?

Strategy Trade-Offs

Good strategy means choosing what not to do.

If We Prioritize...	It Helps Us...	But It May Make It Harder To...
Reliable three-ball scoring	Build and practice one clear plan	Score more than one cycle
Multiple scoring cycles	Increase scoring potential	Stay accurate, avoid traffic, and avoid penalties
A fixed launch site	Aim more consistently	Recover if the site is blocked
Narrow parking-friendly robot	Fit with an alliance partner	Fit the hopper, launcher, battery, and electronics
Simple autonomous movement	Earn reliable early value	Attempt autonomous scoring
Autonomous scoring	Create strong challenge evidence	Finish other systems on time
Penalty avoidance	Protect the score	Drive aggressively or attempt risky plays

Evidence Teams Should Collect

A good strategy should produce evidence. During testing, collect information such as:

- number of balls scored out of three,
- time needed to load, drive, aim, and shoot,
- number of complete scoring cycles in a match-length test,
- accuracy from the first launch site and backup launch site,
- whether two robots can fully fit in base,

- autonomous success rate across several trials,
- driver feedback about control and aiming,
- and rule or penalty concerns noticed during practice.

Main Takeaway

Your team does not need the most complicated robot. Your team needs a strategy that fits your materials, time, drivers, human operator, alliance partner, and evidence from testing.

The best engineering explanation sounds like this:

We chose this strategy because it gave us the best chance to score reliably, avoid penalties, and support our alliance. We accepted this trade-off, and our testing evidence showed this was the right next step.

Human-Centered Controller Mapping Background

How Should the Controller Match the Driver's Intuition?

Robot control is not only a programming problem. It is also a human design problem.

When drivers are calm, they can think carefully about what each button does. When drivers are stressed, rushed, defended, or trying to score before time runs out, they rely more on habit, muscle memory, and previous game experience.

That means a good controller map should feel predictable. The best control scheme is not always the one with the most features. It is the one the driver can use correctly under pressure.

The Big Idea

A controller map is a design decision. Teams should choose controls that match:

- what the robot needs to do,
- what the driver already understands,
- what actions are most urgent,
- what mistakes would be costly,
- and what the team can practice enough to make automatic.

If the driver has to stop and remember the controls during a match, the map is too complicated or not practiced enough.

Human Intuition and Stress

Under pressure, people often fall back on familiar patterns.

Examples:

- If a student has played console games, they may expect the left stick to move and the right stick to turn or aim.
- If a student has used D-pad controls in simple games, they may expect each direction to mean a clear fixed movement.
- If a student has driven remote-control cars, they may expect one stick or trigger to control forward/backward and another control to steer.
- If a student has practiced the same loading and shooting routine many times, they may do it smoothly without needing much verbal coaching.

This is why practice matters. A confusing control map can become manageable with training, but a simple map becomes powerful when the driver has practiced it repeatedly.

What Is the Logitech F310?

The Logitech F310 is a wired USB gamepad commonly used with FTC Driver Stations. It is designed mainly as a PC game controller.

Important features:

Feature	Why It Matters
USB wired connection	Simple and reliable for Driver Station use.
XInput mode	Makes the controller behave like a modern Windows/Xbox-style gamepad.
DirectInput mode	Older PC input mode; useful in some computer games, but usually not the first choice for FTC.
Two analog sticks	Useful for smooth driving, turning, aiming, or mechanism control.
D-pad	Useful for simple fixed-direction driving or step-by-step adjustments.
A/B/X/Y buttons	Useful for discrete actions such as shoot, open trigger, reset, slow mode, or mode change.
Bumpers and triggers	Useful for actions the driver may need while keeping thumbs on the sticks.

For FTC use, the switch on the bottom of the F310 is usually set to **X** for XInput mode. In many FTC setup examples, the driver assigns the controller by pressing **Start + A** for gamepad 1 or **Start + B** for gamepad 2.

How the F310 Relates to Xbox, PlayStation, and PC Controllers

The F310 is not an Xbox or PlayStation controller, but it borrows familiar ideas from both console-style and PC gamepads.

Controller Family	Connection to the F310
PC gamepads	The F310 is officially a PC-style USB gamepad and supports common PC input standards.
Xbox-style controllers	In XInput mode, the F310 uses Xbox-style button names: A, B, X, Y, bumpers, triggers, sticks, Start, and Back. FTC code also commonly uses these names.
PlayStation-style controllers	The physical idea is familiar: two sticks, a D-pad, four face buttons, shoulder controls, and center buttons. The labels are different: PlayStation uses symbols such as triangle, circle, cross, and square instead of A/B/X/Y.
Nintendo-style controllers	The F310 also has four face buttons and a D-pad, but button positions and labels may not match what a Nintendo player expects.

This matters because students may bring different controller habits. A driver who grew up with PlayStation may understand the layout but not the A/B/X/Y names automatically. A driver used to Xbox-style games may recognize the names more quickly.

FTC Code Names

In FTC programs, controller inputs are usually read through names like:

Driver Action	Common FTC Code Name
Left stick forward/back	<code>gamepad1.left_stick_y</code>
Left stick left/right	<code>gamepad1.left_stick_x</code>
Right stick left/right	<code>gamepad1.right_stick_x</code>
D-pad directions	<code>gamepad1.dpad_up</code> , <code>gamepad1.dpad_down</code> , <code>gamepad1.dpad_left</code> , <code>gamepad1.dpad_right</code>
Face buttons	<code>gamepad1.a</code> , <code>gamepad1.b</code> , <code>gamepad1.x</code> , <code>gamepad1.y</code>
Bumpers	<code>gamepad1.left_bumper</code> , <code>gamepad1.right_bumper</code>
Triggers	<code>gamepad1.left_trigger</code> , <code>gamepad1.right_trigger</code>

This means the code should be written in a way the team can explain. For example:

- A might mean shoot.
- B might mean stop shooter.
- Right bumper might open the trigger.
- Left bumper might close the trigger.
- Right trigger might activate slow mode.

The exact map is a team decision.

D-Pad Control

D-pad control uses the cross-shaped direction pad.

What It Helps	What It Makes Harder
Simple directions	Smooth curves
Beginner predictability	Fine speed control
Testing one movement at a time	Natural driving feel
Repeatable short moves	Fast aiming adjustments

D-pad control can be good for first tests because it is clear: up means forward, down means backward, left means turn or slide left, and right means turn or slide right. It can also feel less natural when the driver needs

smooth aiming or tiny corrections.

Joystick Control

Joystick control uses one or both analog sticks.

What It Helps	What It Makes Harder
Smooth driving	Beginner overcorrection
Variable speed	Consistent straight-line movement
Natural video-game feel	Debugging if directions are reversed
Fast aiming corrections	Avoiding sudden full-power movement

Joystick control often feels more like modern console or PC games. It can be powerful, but it may need dead zones, speed scaling, or a precision mode so the robot does not jump or turn too sharply.

Speed and Precision Modes

A robot that is easy to drive at full speed may still be hard to aim. Teams can program control modes to help.

Mode	What It Does	When It Helps
Normal mode	Full driver control range	Moving across the field
Slow mode	Reduces motor power	Aiming, loading, parking, avoiding penalties
Precision turn	Reduces turning speed	Lining up with a launch site
Dead zone	Ignores tiny stick movements	Prevents drift when the stick is near center

Speed modes should be easy to remember. If slow mode is important, put it on a control the driver can reach without looking down.

Mapping Mechanisms

Driving controls should be the easiest to use because the robot is moving while the driver is thinking. Mechanism controls should also match the team's routine.

For a human-loaded ball launcher, the sequence might be:

- Drive to loading position.
- Human operator loads up to three balls.
- Driver moves to launch site.
- Driver aims.
- Driver spins up flywheel.
- Driver opens trigger to release one ball or a controlled set of balls.

- Driver repeats or returns to base.

The controller map should support this sequence.

Robot Action	Human-Centered Mapping Question
Drive	Can the driver move without looking down?
Slow/precision mode	Can the driver aim without accidentally driving too fast?
Spin up flywheel	Is the control hard to press by accident?
Open trigger	Is the control easy to press at the exact moment needed?
Stop shooter	Is there a clear safe stop action?
Reset or unjam	Is this protected from accidental use?

Safety and Error Prevention

Some controls should be easy. Some controls should be deliberate.

Control Type	Good Design Choice
Emergency stop or safe stop	Easy to find and easy to explain.
Driving	Natural and practiced.
Precision mode	Easy to hold or toggle while aiming.
Shooter spin-up	Deliberate enough to avoid accidental activation.
Trigger release	Clear and consistent.
Reset or reverse	Protected from accidental presses.

A good question is: "What is the worst mistake the driver could make with this button map?" Then design the controls to make that mistake less likely.

Training Matters

A good controller map is only good if the driver practices it.

Teams should test:

- Can the driver explain every control without looking?
- Can the driver complete a loading, driving, aiming, and shooting cycle three times?
- Does the driver accidentally press the wrong button?
- Does the driver overcorrect while aiming?
- Does the human operator understand the driver's signals?
- Can another teammate use the same map if the main driver is absent?

Main Takeaway

Controller mapping is a human-centered design decision. The code should match the driver, the robot, the game strategy, and the team's practice time.

The best explanation sounds like this:

We chose this controller map because it matched our driver's intuition, reduced mistakes under pressure, and helped us complete our scoring routine reliably.

Source Notes

This page is based on FTC controller programming conventions and Logitech F310 documentation. Logitech describes the F310 as a PC gamepad with XInput and DirectInput modes. FTC setup examples commonly use the F310 in XInput mode, which behaves like an Xbox-style controller and uses familiar A/B/X/Y button names.